

Documentación de la aplicación

Escáner de Personalidad Pokémon

Documentación funcional y técnica de la aplicación, incluyendo la generación de los perfiles Pokémon, el procesamiento del cuestionario, la comparación vectorial y la obtención del resultado final.

Componente	Descripción
Aplicación	Python + Streamlit
Cuestionario	50 preguntas con 4 respuestas cada una
Perfil del usuario	Vector de 5 dimensiones
Perfiles Pokémon	TF-IDF aplicado a descripciones de PokéAPI
Normalización	Escala relativa de 0 a 10 por dimensión
Comparación	Similitud del coseno
Base	1.025 especies Pokémon
Gimnasio	Votación de tipos a partir de las respuestas
Imagen	Consulta final a PokéAPI

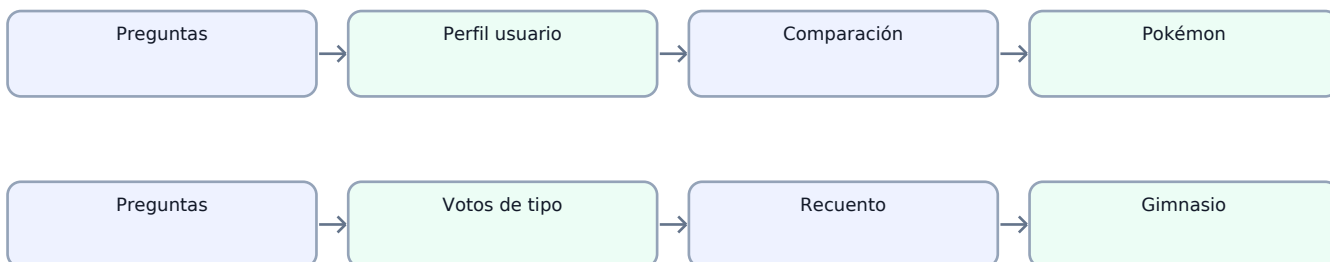
La documentación está redactada para que pueda leerse tanto como introducción para una persona sin conocimientos previos como documentación técnica para alguien familiarizado con Python, procesamiento de texto y vectores.

1. ¿Qué hace la aplicación?

La aplicación presenta un cuestionario de 50 preguntas. A partir de las respuestas obtiene dos resultados:

- un Pokémon cuya representación matemática es la más parecida al perfil del usuario;
- un tipo de gimnasio determinado por la cantidad de votos acumulados para cada tipo Pokémon.

Aunque ambos resultados aparecen juntos al final, se calculan mediante mecanismos diferentes. El Pokémon se obtiene mediante comparación de vectores; el gimnasio mediante recuento de votos.



2. Las dos fases del proyecto

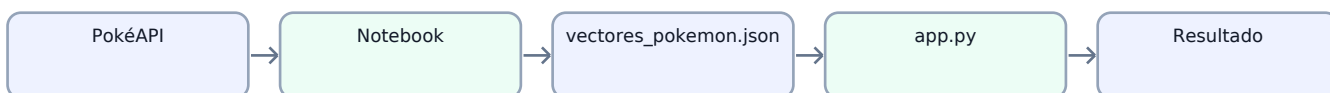
2.1. Preparación de los perfiles Pokémon

Antes de que el usuario realice el test existe un proceso de preparación de datos. El notebook obtiene información de PokéAPI, transforma las descripciones de los Pokémon en números y guarda el resultado en `vectores_pokemon.json`.

- Descargar las especies Pokémon.
- Obtener sus descripciones.
- Aplicar TF-IDF a las palabras seleccionadas.
- Agrupar los pesos en cinco dimensiones.
- Normalizar cada dimensión a una escala de 0 a 10.
- Guardar los perfiles resultantes en JSON.

2.2. Ejecución de la aplicación

Cuando se utiliza la aplicación, ese trabajo ya está realizado. `app.py` carga el JSON, recoge las respuestas, genera el perfil del usuario y lo compara con los perfiles almacenados.



3. Cómo se crean los perfiles Pokémon

3.1. Obtención de las especies

El notebook recorre los identificadores de las especies desde 1 hasta 1025 y realiza una petición a PokéAPI para cada una. En cada respuesta obtiene el nombre y las entradas de descripción disponibles en inglés.

```
url = f"https://pokeapi.co/api/v2/pokemon-species/{i}/"
respuesta = requests.get(url)
datos = respuesta.json()
```

Las diferentes entradas inglesas se combinan para formar un único texto descriptivo de cada Pokémon. El resultado del proceso contiene 1.025 especies.

3.2. ¿Por qué se utilizan las descripciones?

El proyecto no asigna manualmente un perfil completo a cada Pokémon. En su lugar, intenta extraer señales de personalidad a partir del vocabulario que aparece en sus descripciones.

La idea es que determinadas palabras puedan funcionar como indicadores. Por ejemplo, palabras relacionadas con dormir, calma o amabilidad pueden aportar a la dimensión Calmado/Afable; palabras relacionadas con inteligencia o cálculo pueden aportar a Intelectual/Astuto.

4. Las cinco dimensiones de personalidad

El notebook define cinco dimensiones. Cada una tiene un conjunto de palabras clave en inglés.

Dimensión	Palabras clave principales
Agresivo / Audaz	angry, rage, fierce, attack, fight, violent, cruel, destroy, territorial
Calmado / Afable	calm, gentle, peace, sleep, quiet, friendly, heal, soft, kind
Enérgico / Activo	fast, speed, run, quick, energy, active, jump, electric, agile
Intelectual / Astuto	smart, intelligent, brain, mind, wisdom, clever, genius, calculate
Misterioso / Oscuro	dark, shadow, trick, curse, ghost, night, creepy, hide, sinister

Estas palabras constituyen el vocabulario que el proceso TF-IDF utiliza para representar las descripciones desde el punto de vista de las cinco dimensiones.

5. TF-IDF: convertir texto en información numérica

5.1. El problema

Una descripción es texto, pero para comparar Pokémon necesitamos números. Una forma ingenua sería contar palabras. El problema es que una palabra que aparece en casi todas las descripciones no sirve demasiado para distinguir un Pokémon de otro.

5.2. Qué aporta TF-IDF

TF-IDF (Term Frequency-Inverse Document Frequency) asigna un peso a cada palabra según su importancia dentro de un texto y su capacidad para distinguirlo del resto de textos.

De forma intuitiva: si una palabra aparece mucho en un Pokémon pero es poco habitual en el conjunto de descripciones, puede tener un peso elevado. Si aparece prácticamente en todas partes, su peso diferencial será menor.

5.3. Resultado

Después de aplicar TF-IDF se dispone de una matriz numérica. Cada fila representa un Pokémon y cada columna una palabra del vocabulario utilizado.

```
matriz_tfidf = vectorizador.fit_transform(df_pokemon['descripcion'])
```

En este punto todavía tenemos muchas columnas. El siguiente paso consiste en agrupar esas palabras según las cinco dimensiones.

6. De las palabras al perfil Pokémon

6.1. Agrupación por dimensión

Para cada dimensión se toman sus palabras clave y se suman los pesos TF-IDF correspondientes.

```
for dimension, palabras in diccionario_personalidad.items():
    df_pokemon[dimension] = df_vectores[palabras].sum(axis=1)
```

Así, cada Pokémon termina representado por cinco valores. El vector tiene esta estructura:

```
[Agresivo_Audaz, Calmado_Afable, Energico_Activo, Intelectual_Astuto, Misterioso_Oscuro]
```

6.2. Normalización

Los valores obtenidos por la suma todavía dependen de la escala del TF-IDF. Para hacerlos más fáciles de interpretar, cada dimensión se normaliza respecto a su máximo.

```
valor_normalizado = (valor / maximo_de_la_dimension) * 10
```

El máximo de una dimensión recibe 10 y los demás Pokémon reciben un valor proporcional. Por ello, un 10 significa 'máximo relativo dentro de esta dimensión', no una puntuación psicológica absoluta.

6.3. Ejemplos

Pokémon	Agresivo	Calmado	Enérgico	Intelectual	Misterioso
Primeape	5,31	0,68	2,47	2,80	0,00
Alakazam	0,00	0,00	0,00	8,43	0,00
Snorlax	0,00	8,85	0,00	1,45	0,00

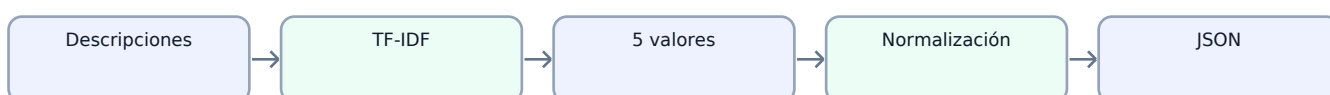
Estos valores muestran cómo un Pokémon puede destacar en una dimensión sin necesidad de que todas las demás tengan valores elevados.

7. El archivo vectores_pokemon.json

Una vez calculados los perfiles, el notebook conserva únicamente la información que necesita la aplicación:

```
['id', 'nombre',
 'Agresivo_Audaz',
 'Calmado_Afable',
 'Energico_Activo',
 'Intelectual_Astuto',
 'Misterioso_Oscuro']
```

La descripción completa ya no es necesaria para el cálculo final, por lo que se exporta una versión reducida. El JSON actúa como puente entre el proceso de preparación de datos y la aplicación web.



8. Cómo se construye el perfil del usuario

8.1. Cada respuesta contiene dos informaciones

En app.py cada opción de respuesta contiene un perfil de cinco valores y una lista de tres tipos Pokémon.

```
{
  "perfil": [4, 0, 3, 0, 0],
  "tipos": ["Fuego", "Lucha", "Roca"]
}
```

El perfil indica cuánto contribuye la respuesta a cada dimensión. Los tipos indican qué tres tipos reciben un voto si el usuario selecciona esa opción.

8.2. Acumulación

La aplicación comienza con un vector de cinco ceros. Cada respuesta añade su propio vector.

```
perfil_personalidad = np.zeros(5)
perfil_personalidad += np.array(datos["perfil"])
```

Con 50 preguntas, el resultado final es la suma de los 50 vectores.

```
Respuesta 1 -> [4, 0, 3, 0, 0]
Respuesta 2 -> [0, 5, 0, 0, 0]
Respuesta 3 -> [0, 0, 0, 5, 2]

Total -> [4, 5, 3, 5, 2]
```

El ejemplo anterior utiliza únicamente tres respuestas para mostrar el mecanismo. En la aplicación real se acumulan las 50.

9. Cómo se encuentra el Pokémon

9.1. Mismo espacio de cinco dimensiones

Aquí está la idea central del proyecto: tanto el usuario como cada Pokémon terminan representados por cinco números y, por tanto, pueden compararse dentro del mismo espacio.

```
Usuario:
[A, C, E, I, M]

Pokémon:
[A, C, E, I, M]
```

9.2. Similitud del coseno

La aplicación utiliza la similitud del coseno. Esta medida compara la orientación de dos vectores. Conceptualmente, si los dos perfiles apuntan en direcciones parecidas, su similitud será alta.

$$\cos(\theta) = (A \cdot B) / (|A| \cdot |B|)$$

La ventaja es que la comparación se centra en la distribución relativa de las dimensiones. No se limita a comprobar si los cinco números son grandes o pequeños.

9.3. Comparación contra toda la base

```
coincidencias = cosine_similarity(
    vector_usuario,
    df_pokemon[columns dimensiones].values
)[0]

mejor_indice = np.argmax(coincidencias)
mejor_pokemon = df_pokemon.iloc[mejor_indice]
```

El cálculo genera una similitud para cada Pokémon. `np.argmax` localiza la posición del valor máximo y esa fila se convierte en el Pokémon ganador.

10. Ejemplo de comparación

El notebook incluye una prueba con un vector de usuario:

```
puntuacion_usuario = [1.0, 9.5, 2.0, 7.0, 0.5]
```

Este vector representa un perfil con una fuerte presencia de Calmado/Afable e Intelectual/Astuto. Al compararlo con todos los Pokémon, el notebook obtiene el siguiente orden:

Posición	Pokémon	Similitud
1	Abra	0,971014
2	Slowking	0,968992
3	Herdier	0,960600

El ejemplo demuestra que el resultado se obtiene a partir de la geometría de los vectores, no mediante una regla que diga directamente qué Pokémon corresponde a una respuesta concreta.

11. El gimnasio funciona de forma diferente

El gimnasio no se calcula mediante TF-IDF ni mediante similitud del coseno.

11.1. Votos

Cada respuesta tiene tres tipos asociados. Cuando se selecciona una respuesta, se suma un voto a cada uno.

```
for tipo in datos["tipos"]:  
    puntuacion_tipos[tipo] += 1
```

Al finalizar el cuestionario existen contadores para los tipos disponibles. El tipo que acumula más votos se utiliza como gimnasio.



11.2. Dos caminos a partir de una respuesta

Una misma respuesta puede influir en los dos resultados:

- su vector modifica el perfil utilizado para encontrar el Pokémon;
- sus tres tipos modifican el resultado del gimnasio.

Por tanto, no es correcto decir que el gimnasio se obtiene a partir del Pokémon ganador. Son dos cálculos independientes que parten de las mismas respuestas.

12. Ejemplo de una respuesta

```
A) Tomo el mando, decido la hora y compro los billetes del tirón  
  
perfil = [4, 0, 3, 0, 0]  
tipos = [Dragón, Fuego, Lucha]
```

Esta opción contribuye de la siguiente manera:

- 4 puntos a Agresivo/Audaz.
- 3 puntos a Enérgico/Activo.
- 1 voto para Dragón.
- 1 voto para Fuego.
- 1 voto para Lucha.

No selecciona directamente ningún Pokémon. Su efecto se incorpora al resultado global después de procesar todas las respuestas.

13. Flujo de la aplicación Streamlit

13.1. Formulario

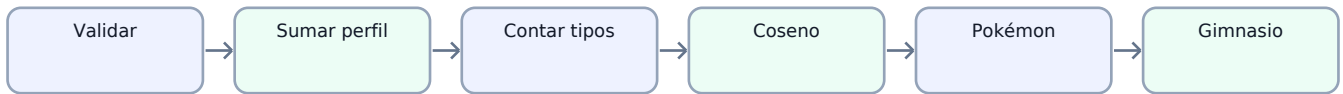
Streamlit genera el formulario y presenta las 50 preguntas como controles de selección. Las opciones comienzan sin seleccionar.

13.2. Validación

Al pulsar el botón de análisis, la aplicación comprueba si alguna pregunta sigue sin responder. Si falta alguna, se muestran las preguntas pendientes y se detiene el análisis.

13.3. Cálculo

Si todas están respondidas, se ejecuta la secuencia completa:



13.4. Resultado

Después se calculan los porcentajes del perfil del usuario, se identifican los rasgos principales y se prepara la información que se mostrará en pantalla.

14. Consulta final a PokéAPI

El perfil matemático del Pokémon ya está en el JSON. Sin embargo, para mostrar su ilustración, app.py realiza una nueva petición a PokéAPI utilizando el identificador del Pokémon ganador.

```
url_api = f"https://pokeapi.co/api/v2/pokemon/{mejor_pokemon['id']}"
respuesta_api = requests.get(url_api).json()
```

De la respuesta se busca la ilustración oficial. Esta consulta no decide el resultado; únicamente aporta información visual después de que el algoritmo ya haya elegido el Pokémon.

15. Porcentajes y texto final

El vector acumulado del usuario se convierte en porcentajes relativos:

```
total_puntos = np.sum(perfil_personalidad)
porcentajes = (perfil_personalidad / total_puntos) * 100
```

Por ejemplo, si el vector fuera [20, 40, 10, 25, 5], el total sería 100 y los porcentajes serían 20 %, 40 %, 10 %, 25 % y 5 %.

La aplicación ordena las dimensiones para identificar el rasgo principal y el secundario. El resultado final combina la información del perfil, la afinidad con el Pokémon y el gimnasio.

16. Arquitectura técnica completa



La arquitectura puede entenderse como dos circuitos.

Circuito de preparación

- PokéAPI proporciona las descripciones.
- El notebook las procesa.
- TF-IDF convierte el texto en pesos.
- Las palabras se agrupan en cinco dimensiones.
- Las dimensiones se normalizan.
- Los perfiles se guardan en JSON.

Circuito de ejecución

- app.py carga el JSON.

- El usuario contesta 50 preguntas.
- Se suma el vector de cada respuesta.
- Se cuentan los tipos.
- Se compara el vector del usuario con todos los Pokémon.
- Se selecciona la mayor similitud.
- Se consulta PokéAPI para la imagen.
- Se presenta el resultado.

17. Tecnologías utilizadas

Tecnología	Función dentro del proyecto
Python	Lenguaje principal.
Pandas	Gestión de tablas y lectura del JSON.
NumPy	Vectores y operaciones numéricas.
scikit-learn	TF-IDF y similitud del coseno.
Requests	Peticiones HTTP a PokéAPI.
Streamlit	Interfaz web del cuestionario.
PokéAPI	Fuente de información Pokémon e imágenes.
JSON	Almacenamiento de los perfiles ya calculados.

18. Qué significa técnicamente el resultado

La palabra 'personalidad' describe el propósito lúdico de la aplicación. Desde el punto de vista técnico, el sistema realiza una clasificación basada en características.

El perfil Pokémon se obtiene de descripciones textuales mediante TF-IDF y un conjunto de palabras clave. El perfil del usuario se obtiene mediante reglas asignadas a las respuestas. La similitud del coseno determina qué perfil Pokémon tiene una orientación más parecida.

Por tanto, el resultado es explicable: se puede seguir el recorrido desde las palabras de una descripción hasta sus cinco valores y, posteriormente, desde las respuestas del usuario hasta su vector final.

19. Limitaciones

- Dependencia de palabras clave: si una descripción expresa una característica con palabras que no están en el vocabulario, esa señal no contribuye a la dimensión.
- Dependencia del inglés: las descripciones utilizadas y las palabras clave están basadas en inglés.
- TF-IDF no interpreta frases como una persona: trabaja con la importancia estadística de términos.
- Normalización relativa: un 10 significa máximo dentro de la dimensión utilizada para normalizar.
- Perfiles contruidos de distinta forma: Pokémon mediante texto; usuario mediante reglas de las respuestas.
- Dependencia de PokéAPI: la imagen final requiere una petición externa.

20. Mejoras técnicas posibles

Prioridad	Mejora	Objetivo
Alta	Separar lógica de negocio e interfaz	Facilitar pruebas y mantenimiento.
Alta	Añadir timeout y manejo de errores HTTP	Hacer más robusta la consulta a PokéAPI.
Alta	Validar el JSON al cargarlo	Detectar datos o columnas incorrectos.
Media	Externalizar las palabras clave	Modificar el modelo sin editar el programa principal.
Media	Versionar la generación de perfiles	Saber con qué configuración se creó cada JSON.
Media	Añadir pruebas unitarias	Comprobar acumulación, coseno y selección.
Media	Mostrar Top 3	Dar más contexto al resultado.

21. Explicación breve para una presentación

Una explicación clara del proyecto podría ser la siguiente:

Primero, obtenemos las descripciones de los Pokémon desde PokéAPI. Después aplicamos TF-IDF para transformar el texto en pesos numéricos. Agrupamos esos pesos según cinco dimensiones de personalidad y normalizamos cada dimensión de 0 a 10. De esta forma, cada Pokémon queda representado mediante un vector de cinco valores que guardamos en un JSON.

Después, el usuario responde 50 preguntas. Cada respuesta contiene un vector de cinco valores y tres tipos Pokémon. Los vectores se suman para obtener el perfil del usuario y los tipos se acumulan como votos.

Finalmente, comparamos el vector del usuario con todos los vectores Pokémon utilizando similitud del coseno. El Pokémon con mayor similitud es el resultado. De forma independiente, el tipo con más votos determina el gimnasio. La aplicación consulta entonces PokéAPI para mostrar la imagen del Pokémon.

22. Mapa de responsabilidades del código

Parte	Responsabilidad
Notebook: extracción	Descargar especies y descripciones desde PokéAPI.
Notebook: TF-IDF	Convertir descripciones en pesos numéricos.
Notebook: dimensiones	Sumar palabras por cada dimensión.
Notebook: normalización	Transformar los valores a escala 0-10.
Notebook: exportación	Crear vectores_pokemon.json.
app.py: formulario	Mostrar las 50 preguntas.
app.py: validación	Comprobar que no falten respuestas.
app.py: perfil	Sumar los vectores de las respuestas.
app.py: tipos	Acumular los votos de tipos.
app.py: coseno	Comparar al usuario con todos los Pokémon.
app.py: PokéAPI	Obtener la imagen del Pokémon ganador.
app.py: resultado	Mostrar perfil, afinidad y gimnasio.

23. Conclusión

El funcionamiento completo se puede resumir como una transformación progresiva:



La idea central del proyecto es que los textos de los Pokémon y las respuestas de los usuarios terminan representados en el mismo espacio de cinco dimensiones. Esto hace posible utilizar una técnica matemática, la similitud del coseno, para encontrar el perfil Pokémon más parecido.

El gimnasio constituye un segundo sistema, más sencillo: cada respuesta aporta tres votos y el tipo con mayor puntuación determina el resultado.

De esta forma, la aplicación combina procesamiento de texto, representación vectorial, aprendizaje automático basado en TF-IDF, operaciones numéricas, una interfaz web y una API externa en un flujo único y comprensible.